Mk33: A Fully Specified Recipe and Reference Implementation for Training Long-Horizon Self-Improving Agents

Dulguun Enkhtsatsral
New York University Shanghai
de2318@nyu.edu

Abstract

AREX-2 shows that a 27B-parameter agent trained by imitation on long-horizon improvement trajectories can keep refining a solution for hours. The paper describes its training recipe in prose and releases only evaluation code, leaving open the thresholds for filtering environments and accepting trajectories, the definition of the "no-progress" turns excluded from the loss, the agent's tool set, and what happens when a multi-hour run exceeds the context window. We present Mk33, a complete specification of this recipe with an open, tested reference implementation. It contributes a noise-aware environment filter with decontamination and an isolated, audited scorer; Round Ledger Compaction (RLC), a context manager that keeps every round, including failures, in a structured ledger and yields training windows identical to the contexts seen at test time; and Progress-Aware Loss Masking (PALM), deterministic rules that zero out redundant, null, and polling turns and give full credit to turns that lead to score improvements. Because the full recipe needs a GPU cluster, we design every experiment so that it also runs at small scale on one free GPU or a laptop: the same scaffold, metrics, and statistics, with a context window shrunk so that runs still outgrow it. In a memory study at equal cache size on two 3B models, attention-based KV-cache eviction (H2O) had the lowest loss in every setting and stayed within 0.05 to 0.07 nats of the full cache while keeping half the context, while BM25 retrieval was never distinguishable from plain truncation. We also introduce a re-try rate that measures forgetting directly, as the share of rounds that repeat a known dead end. In five paired runs of up to 40 rounds with a 7B model on a laptop CPU, RLC needed 2.4 to 9.6 times less time per model call than truncation at a similar prompt length, which we trace to its frozen prefix: on average 72 to 87% of each RLC prompt repeats the start of the previous one, against 39 to 52% under truncation, so the inference server can reuse its cache. RLC finished all 40 rounds in three runs and truncation in none. Whether RLC also leads to better solutions is not established: across the three runs without a confound, each mode scored higher at least once, and the gaps trace to failures unrelated to memory. In 8-round grids with 3B and 7B models on one GPU, the memory methods were not distinguishable, because in most runs the context overflows late or never. The comparison of PALM with AREX-2's masking is specified and implemented but not yet run.

1 Introduction

An agent that can improve its own solution over many rounds converts extra test-time compute into better results. AREX-2 [19] frames this as self-improvement through test-time scaling and splits the gain into two capabilities: *reflection*, which sets the mean gain per productive round, and *long-horizon execution*, which sets how many rounds remain productive. Trained only by imitation on teacher trajectories from machine-learning engineering and competitive-programming environments, AREX-2 reports 81.8 on MLE-bench Lite and 70.7 on Frontier-CS, along with gains on deep-research benchmarks that provide no score feedback.

The result is useful but difficult to build on. The public repository contains evaluation runners only. The paper specifies its pipeline qualitatively: environments are kept when a baseline scores "well below the reference"; the loss skips "near-duplicate turns"; trajectories are accepted when the final score is "high." Two

questions receive no answer: how a run that lasts up to 12 hours fits in a finite context window, and which teacher model generates the data.

This report does not claim a new capability at AREX-2's scale. Its goal is to make the recipe reproducible and testable. For each component AREX-2 leaves open, we give a precise specification, a rationale, a configuration key, and an ablation that tests whether the choice matters, and we release a reference implementation so that each specification can be checked against working code.

Testing the recipe at its intended scale requires thousands of multi-hour rollouts and full fine-tuning of a 27B model. We therefore also ask a narrower question that can be answered with modest compute: *do the mechanisms work when the problem they address is reproduced at small scale?* The defining difficulty of long-horizon runs is that the history outgrows the context window. With an 8K-token window instead of 128K, a run of a few dozen rounds on a 1.5B to 3B model reaches the same situation, so RLC, PALM, and their alternatives can be compared under controlled conditions on one free GPU (Section 9).

Contributions.

1. **Environment specification** (Section 3): baseline-to-reference normalization, a noise-aware headroom filter, benchmark decontamination, and scorer isolation with a process audit against reward hacking.
2. **Round Ledger Compaction** (Section 4): a context manager for multi-hour runs that keeps a structured record of every round, freezes the context prefix between compactions, and is shared verbatim by the agent and the training data pipeline.
3. **Progress-Aware Loss Masking** (Section 5): formal rules for zero-weight turns and a provenance set of full-credit turns, computed deterministically from trajectory logs.
4. **A complete recipe and tested implementation** (Sections 6 and 7): acceptance rules, every training hyperparameter, a configuration file per ablation, and a test suite that checks the properties the method depends on, including that training contexts reproduce test-time contexts exactly.
5. **A memory study at equal cache size** (Section 9.3): text-level forgetting (truncation, BM25 retrieval) against KV-cache eviction (random, streaming, key-norm, H2O) on two 3B models, with paired bootstrap intervals and fused Triton kernels for the eviction step.
6. **Small-scale experiments on free hardware** (Section 9): task success under each memory method, 40-round runs on a laptop that show a large speed advantage for RLC from prefix-cache reuse, a catalogue of how small agents fail in long runs, and a new *re-try rate* metric for forgetting. All raw runs and the scripts that compute every number are released.

2 Background and Gap Analysis

The AREX-2 framework. In round t the agent submits a solution y_t and receives feedback f_t: a score plus logs, errors, and timings. The expected total improvement over T rounds is

$$\mathbb{E}[\Delta_{\text{total}}] = \sum_{t=1}^{T} r_t \approx \bar{r} \cdot T^*, \tag{1}$$

where $\bar{r}$ is the mean gain per productive round and T^* is the number of productive rounds. The AREX-2 pipeline has four stages: (i) build environments from GitHub repositories and online judges; (ii) generate long teacher trajectories that keep every round, including regressions; (iii) accept whole trajectories by final score and process conformance; and (iv) fine-tune Qwen3.8-27B [21] with loss only on turns that make progress, mixed with unchanged deep-research data from the AREX recipe [12].

Table 1: Components of the AREX-2 recipe that are unspecified, and where Mk33 specifies them.

Component	Stated in AREX-2	Missing	Here
Environment filter	Baseline "well below the reference"	Numeric margin; score noise	§3.2
Score scale	Graded scores "where possible"	Normalization across metrics	§3.1
Contamination	Not discussed	Overlap with evaluation tasks	§3.3
Process conformance	Process must conform	Concrete checks	§3.4
Agent scaffold	Tool calls, submission, feedback	Tool set; scaffold code	§4.1
Context management	Failures stay in context; 12 h runs	Behavior on overflow	§4
Loss masking	No loss on polling, empty calls, near-duplicates	Detection rules; "progress"	§5
Acceptance	Best score above a relative threshold	Values; minimum rounds	§6.1
Teacher model	"A teacher model"	Identity; sensitivity	§6.2
Training	Base model; data mix	All hyperparameters	§6.3
Released code	Evaluation only	Synthesis, scaffold, training	§7

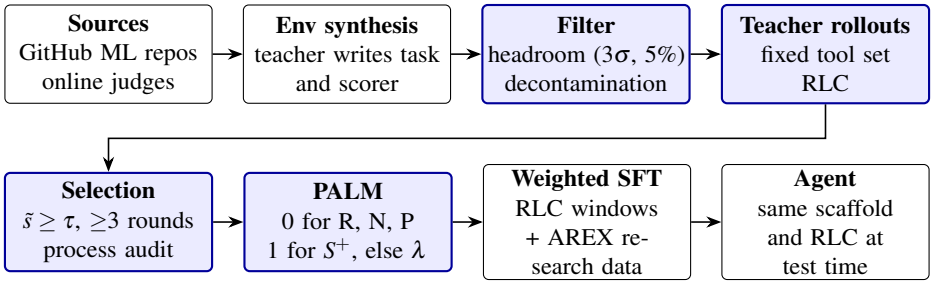


Figure 1: The Mk33 pipeline. Shaded stages contain components that are new relative to AREX-2; the others follow AREX-2 and are now fully specified. The same scaffold and context manager are used for teacher rollouts, for building training windows, and at test time.

What is unspecified. Table 1 lists each component that the paper or its repository leaves open, and where this report addresses it. We add one further check that AREX-2 does not run. Its claim that gains transfer across domains rests on holding the deep-research data fixed. That design does not separate the effect of iterative trajectory structure from the effect of additional agentic training tokens. Ablation A7 (Section 8) adds a token-matched control.

3 Environments

Mk33 uses the same two environment sources as AREX-2. For *ML repositories*, a candidate must have a permissive license, a runnable entry point, a quantitative metric already computed in its code, and a full run under 2 GPU-hours on one 80 GB GPU. A teacher model reads a digest of the repository and drafts a task statement asking for the metric to be improved, a deterministic held-out split, a scoring script, a baseline command, and a reference command. For *online-judge problems*, we prefer graded scoring: optimization problems scored against the best known value, or problems with many hidden tests scored by the fraction passed.

3.1 Normalized score

Raw metrics differ in scale and direction. For environment e with baseline score b_e (a simple solution such as a default configuration, majority class, or brute force) and reference score $s_{\mathrm{ref},e}$ (the mean of k seeded runs),

every score is mapped to

$$\tilde{s} = \frac{s - b_e}{s_{\mathrm{ref},e} - b_e}, \tag{2}$$

with all three quantities negated when lower is better. The baseline maps to 0 and the reference to 1. Run-to-run noise on this scale is $\varepsilon_e = \sigma_e / |s_{\mathrm{ref},e} - b_e|$, where σ_e is the standard deviation of the reference runs; ε_e is used below to decide whether a change in score is real.

3.2 Noise-aware headroom filter

AREX-2 requires the baseline to score "well below" the reference. We run the reference $k = 3$ times and keep an environment only if

$$s_{\mathrm{ref},e} - b_e \;\geq\; \max\bigl(3\,\sigma_e,\; 0.05\,|s_{\mathrm{ref},e}|\bigr). \tag{3}$$

The $3\sigma_e$ term ensures that improvement on the normalized scale is a real signal rather than seed noise. The 5% term rejects environments whose headroom is statistically significant but too small to support many productive rounds. For judge problems scored by pass fraction we instead require $b_e \leq 0.6$ and $s_{\mathrm{ref},e} \geq 0.95$. Both margins are swept in ablation A4.

3.3 Decontamination

GitHub ML repositories can overlap with Kaggle competitions in MLE-bench [2], and judge problems can overlap with Frontier-CS [14]. An environment is dropped if, against any evaluation task, it shares a dataset (normalized URL, normalized name, or file hash), shares a competition or problem identifier, or has 13-gram overlap of at least 50% between task statements, following the n-gram check of Brown et al. [1]. We report the number of environments each rule removes. Ablation A6 measures how much of the final score depends on this step.

3.4 Scorer isolation and process audit

AREX-2 accepts trajectories only if the process conforms to the task requirements, without stating how this is checked. Verifiable environments invite reward hacking [24], so we enforce conformance by construction and then audit it.

1. The held-out split and scoring script live in a scorer directory that is never mounted in the agent's sandbox. The agent sees only a public development split and a `submit` tool.
2. Before every run the scorer verifies the SHA-256 hash of its own script and scores a frozen copy of the solution. In container mode the solution is mounted read-only and the scorer has no network access.
3. For judge problems, 10% of hidden tests are canaries generated after the trajectory is collected. A solution whose pass rate on regular tests exceeds its pass rate on canaries by more than 0.2 is flagged as overfit.
4. Rule checks flag reads of paths outside the workspace or inside the scorer directory, hidden-test outputs embedded as literals in the final solution (one output of at least 32 characters, or three of at least 8), and fetches of the source repository's reference solution.
5. An optional LLM auditor reviews the final diff for metric gaming, such as changing the evaluation split.

A trajectory flagged by any of steps 3 to 5 is rejected.

Table 2: The fixed tool set.

Tool	Behavior
`bash`	Shell command in the sandbox; 10-minute timeout; output truncated to about 8K tokens, keeping head and tail.
`read_file`, `edit_file`	Inspect and change files. Edits require a unique match and return a unified diff, which PALM uses for provenance.
`run_job`, `wait_job`	Start a long command; block until it finishes or a timeout (at most 30 minutes) passes, returning only new log lines.
`web_search`, `web_fetch`	Find papers, documentation, and libraries. Disabled where a benchmark forbids it.
`submit`	Score the current solution. Requires a `hypothesis` and a one-line `lesson`. Ends the round.
`answer`	Final answer for research tasks.

4 Agent Scaffold and Round Ledger Compaction

4.1 Tool set

AREX-2 does not list its tools. We fix one set for all environments so that trajectories share an action space (Table 2). The blocking `wait_job` tool is a deliberate change: in AREX-2, polling turns are generated and then masked out of the loss; a blocking wait removes most of them at the source, which saves context and wall-clock time.

4.2 The overflow problem

A 12-hour run with hundreds of tool calls produces far more tokens than a context window holds. AREX-2 states that failures and regressions remain in the context, but not what happens when it is full. Truncating the oldest turns removes the early failures the model is meant to learn from; free-form summarization loses scores and error signatures.

4.3 Round ledger

The scaffold keeps a *round ledger* with one entry per submission. Most fields are filled automatically: round number, elapsed time, a diff summary of the round's edits, raw score, normalized score, change relative to the best so far, and the first error line of a failed run. The agent writes only two fields, the `hypothesis` and `lesson` arguments of `submit`. Each new entry is also appended to the `submit` observation, so the model sees it immediately. When the ledger exceeds a token cap, its oldest non-best entries are merged into one line per strategy family, recording their rounds, best score, number of failures, and recent lessons. No round is deleted, so the AREX-2 principle that failures stay visible holds for runs of any length. For research tasks without a scorer, entries record a candidate answer, a confidence, and evidence URLs instead of a score.

4.4 Compaction with a frozen prefix

The context consists of a *prefix* (system prompt, task statement, skills, current best, and the rendered ledger) followed by the verbatim turns of recent rounds. Algorithm 1 gives the procedure. The prefix is rendered at the start of a run and re-rendered only when a compaction occurs. Between compactions the context therefore grows only by appending turns. This has two consequences. First, key-value caches remain valid across turns. Second, and more important for training, every segment between two compactions is a single sequence that covers all of its turns, which makes the training windows of Section 4.5 exact rather than approximate.

Algorithm 1 Round Ledger Compaction: building the context for the next turn

Require: window W, threshold θ, rounds kept K, ledger cap L; current round r

if prefix is unset **then** prefix $\leftarrow$ RENDER(pinned, ledger)
$C \leftarrow$ prefix $\|$ visible turns
if $|C| \leq \theta W$ **then return** C ▷ no event: context only grew
$c' \leftarrow r - K + 1$
if $c' \leq c$ **and** $|\text{ledger}| \leq L$ **and** $|C| \leq W$ **then return** C ▷ nothing to compact yet
$c \leftarrow \max(c, c')$ ▷ turns from rounds $< c$ are now ledger-only
if $|\text{ledger}| > L$ **then** merge oldest non-best entries per strategy family
prefix $\leftarrow$ RENDER(pinned, ledger) ▷ includes a note that rounds $1..c-1$ were compacted
$C \leftarrow$ prefix $\|$ visible turns
while $|C| > W$ **and** more than one visible turn **do** drop the oldest visible turn
record a compaction event; **return** C

Defaults are $W = 128$K tokens, $\theta = 0.75$, $K = 2$, and $L = 16$K tokens. The final loop is a guard for a single round that alone exceeds the window.

4.5 Identical contexts at training and test time

A model trained on uncompacted trajectories but run with compaction sees a context distribution it was never trained on. We avoid this by construction: one context-building component is used by the live agent and by the offline training replay. Tool-call identifiers are deterministic (`call_`i for turn i), so the replay emits the same messages byte for byte. The teacher runs with RLC during collection, so its decisions are conditioned on the same views. To build training data, each accepted trajectory is replayed; every compaction event starts a new window whose prefix is the compacted context at that moment, and the turns that follow are appended until the next event. Loss is applied only to assistant turns inside the window, weighted by PALM. The reference implementation tests this property directly (Section 7).

5 Progress-Aware Loss Masking

AREX-2 applies loss to "decisions that move the solution forward" and none to "repeated polling, calls that return no new observation, and near-duplicate turns," without saying how these are detected. PALM turns these descriptions into deterministic rules over the trajectory log, so masks can be reproduced and audited.

Setup. A trajectory has assistant turns $a_1, \ldots, a_N$, each consisting of reasoning and one tool call, followed by an observation o_i. After each turn the scaffold records a hash h_i of the workspace. Let B_t be the best normalized score after round t, with $B_0 = 0$ (the baseline). A submission at round t is *improving* if $\tilde{s}_t > B_{t-1} + \varepsilon_e$.

5.1 Zero-weight turns

A turn receives weight 0 if any of the following holds.

- **Redundant**, $R(i)$: some earlier turn j in the last $W_R = 20$ turns has a near-identical canonical action (tool name and arguments with whitespace normalized; Jaccard similarity of word 5-gram shingles ≥ 0.9), and the workspace has not changed since turn j, including by turn i itself. Re-running a training

script after an edit is therefore a new experiment, not a duplicate. Status checks are excluded from R and handled by P, because a repeated status check can return new information.

- **Null observation**, $N(i)$: after normalization (removing timestamps, process IDs, durations, ANSI codes, and progress-bar lines) the observation is empty with the workspace unchanged, or identical to the last observation from the same tool on the same target with the workspace unchanged.
- **Polling**, $P(i)$: a status-only call (for example `wait_job` returning on timeout, `ps`, `nvidia-smi`, or `tail` on a log) that adds no new log lines since the last status call on the same target.

System messages, observations, and retrieved documents also receive no loss, as in AREX-2.

5.2 Full-credit turns

A turn joins the provenance set S^+ if any of the following holds.

1. *Surviving edit*: it is an `edit_file` turn and at least one of its diff hunks appears line for line (ignoring blank lines and trailing whitespace) in the solution scored at a later improving submission. A deletion-only hunk survives if the removed block is still absent. This approximates `git blame` without requiring repository history.
2. *Improving submission*: it is the `submit` call of an improving round.
3. *Supporting evidence*: it is a read, search, fetch, or inspection turn within 50 turns before a turn from rule 1 or 2, and the later turn uses it: the two share a file path, URL, or identifier, or an 8-gram of the observation appears in the later turn's reasoning. Paths and identifiers that occur in the task statement are excluded, since every file the task names would otherwise qualify. Only one hop is followed.
4. *Recovery*: it is the first edit after a regression ($\tilde{s}_t < B_{t-1} - \varepsilon_e$) or a failed submission, it touches the file named in the error if one is named, and the agent regains at least B_{t-1} within $M = 3$ rounds. This credits diagnosis and repair even when the repaired code is later replaced.

5.3 Weights and loss

All remaining turns receive weight λ. These are plausible steps that did not lead to a gain, such as a reasonable experiment that failed. The AREX-2 description implies $\lambda = 0$ for some of them, yet trajectories are accepted without filtering rounds so that the model learns to explore. We therefore treat λ as an open question, default to $\lambda = 0.5$, and sweep it in ablation A1:

$$w_i = \begin{cases} 0 & \text{if } R(i) \vee N(i) \vee P(i), \\ 1 & \text{if } i \in S^+, \\ \lambda & \text{otherwise.} \end{cases} \tag{4}$$

The loss is a weighted token-level cross-entropy normalized by the weighted token count, so that long trajectories do not dominate a batch:

$$\mathscr{L}(\theta) \;=\; -\frac{\sum_i w_i \sum_{k \in a_i} \log p_\theta(x_k \mid x_{<k})}{\sum_i w_i \, |a_i|}. \tag{5}$$

Reasoning tokens inside a turn share the turn's weight.

5.4 Checking the rules

Heuristic rules can be wrong, so we measure them before training. Two annotators label 500 turns sampled from 50 trajectories as *no progress*, *neutral*, or *progress*. We report precision and recall of the zero-weight rules and of S^+ against these labels, Cohen's κ between the rules and the adjudicated labels, and κ between the two annotators. If the rules agree with the labels less than the annotators agree with each other, thresholds are revised before any training run.

6 Trajectory Selection and Full-Scale Training Recipe

6.1 Acceptance rule

As in AREX-2, whole trajectories are accepted or rejected and individual rounds are never filtered. A trajectory is accepted if (i) its best normalized score satisfies $\tilde{s}_{\text{best}} \geq \tau_d$, with $\tau_{\text{ML}} = 0.9$ and $\tau_{\text{judge}} = 0.95$; (ii) it has at least 3 scored rounds; (iii) at least one round after the first is improving, so every accepted trajectory shows iteration; and (iv) the process audit raised no flag. At most two trajectories are kept per environment, preferring higher best scores, so that easy environments do not dominate. Ablation A3 sweeps $\tau \in \{0.7, 0.9, 1.0\}$.

6.2 Teacher model

AREX-2 does not name its teacher, although results may depend heavily on it. We select the teacher with a pilot: each candidate runs on 50 held-out environments, and we choose the one with the most accepted trajectories per GPU-hour. We prefer an open-weight teacher so that trajectories can be redistributed under its license. The teacher and version will be reported with results. Ablation A5 retrains with the runner-up teacher.

6.3 Data and hyperparameters

Table 3 lists the planned data scale and the full-scale training configuration; the small-scale configuration used in Section 9.5 is given there. These are proposed defaults chosen to match common practice for full-parameter supervised fine-tuning at this scale; they will be tuned on a validation split of 100 environments, and any changes will be reported.

7 Reference Implementation

We release the implementation as a standalone Python package that runs on Linux, macOS, and Windows. The package has seven modules: `envs` (normalization, filter, decontamination, scorer, audit, synthesis), `scaffold` (tools, sandboxes, ledger, RLC, retrieval memory, shared context builder, agent loop), `palm` (rules, provenance, weights, validation), `selection`, `train` (window construction and the weighted loss of Equation (5)), `metrics`, and `kv` (KV-cache eviction policies and kernels for local models). A command-line tool runs each pipeline stage, and every threshold in this report is a configuration value. Each ablation in Table 4 is a configuration file that inherits the defaults and overrides only what it changes.

Tests. The suite (139 tests, run on Linux, macOS, and Windows in continuous integration) checks the properties the method depends on: normalization for both metric directions; each filter, decontamination, and audit rule; that the scorer refuses to run when its script hash changes; that ledger compaction never drops a round and never merges the best entry; that RLC contexts are append-only between events; each PALM rule,

Table 3: Planned data scale and training configuration (proposed defaults, not tuned values).

Item	Value
Environments after filtering	2,000 (1,200 ML, 800 judge)
Teacher rollouts	4 per environment, temperature 0.7
Rollout budget	12 h (ML) or 5 h (judge); at most 400 tool calls
Deep-research data	AREX recipe data, unchanged; default token ratio 1:1
Base model	Qwen3.8-27B (as in AREX-2)
Method	Full-parameter SFT with PALM weights; no reinforcement learning
Context	128K tokens, RLC windows (Section 4.5)
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1
Learning rate	Peak 10^{-5}, 3% linear warmup, cosine decay to 10^{-6}
Epochs; global batch	2; 64 windows
Gradient clipping	1.0
Precision; parallelism	bf16; ZeRO-3 with sequence parallelism
Seeds	3 for the main model, 1 per ablation

including that a re-run after an edit is not redundant and that a status check with new lines is not polling; and the acceptance rule and process metrics. An end-to-end test runs a scripted agent with a context window small enough to force compaction, computes PALM weights, applies the acceptance rule, builds training windows, and asserts that *every context the agent sent to the model is a prefix of the training window that covers that turn*, the property Section 4.5 relies on.

Training at small scale. The trainer supports LoRA [10], float16, and SDPA attention, so that the experiments of Section 9.5 fit on one 16 GB GPU. Because most tokens in a window are context with weight zero, the loss is computed only at positions that predict a weighted token: with a batch of one, only those hidden states pass through the language-model head. On a 1.5B model with an 8K window this reduces the logits from about 5 GB to a few hundred MB; tests confirm that the loss and every parameter gradient equal those of the full computation. Two further details keep training and inference contexts identical: the chat template receives exactly the tool schemas the agent sends (six for the bench, in the same order), and tool-call arguments pass through the same conversion before templating in training and in the local inference client. A test runs the whole small-scale path on CPU with a tiny randomly initialized model: teacher runs, selection, PALM and baseline windows, LoRA training, merging, and generation with the merged model.

We release the implementation as a standalone Python package; the benchmarks in Section 8 are run with the AREX-2 evaluation harness, released separately by its authors, so that scores are directly comparable. The package has six modules: `envs` (normalization, filter, decontamination, scorer, audit, synthesis), `scaffold` (tools, sandboxes, ledger, RLC, shared context builder, agent loop), `palm` (rules, provenance, weights, validation), `selection`, `train` (window construction and the weighted loss of Equation (5)), and `metrics`. A command-line tool runs each pipeline stage, and every threshold in this report is a configuration value. Each ablation in Table 4 is a configuration file that inherits the defaults and overrides only what it changes.

The test suite checks the properties the method depends on, including: normalization for both metric directions; each filter and decontamination rule; that the scorer refuses to run when its script hash changes; each audit rule; that ledger compaction never drops a round and never merges the best entry; that RLC contexts are append-only between events; each PALM rule, including that a re-run after an edit is not redundant and that a status check with new lines is not polling; and the acceptance rule and process metrics. An end-to-end test runs a scripted agent on a small environment with a context window small enough to force compaction, then computes PALM weights, applies the acceptance rule, builds training windows, and asserts that *every*

Table 4: Ablations. Each changes one component and keeps the rest at the defaults.

ID	Question	Variants
A1	Does PALM beat simpler masks?	All assistant tokens; AREX-style mask (zero-weight rules only, $\lambda = 1$); LLM-judge mask; PALM with $\lambda \in \{0, 0.25, 0.5, 1\}$
A2	Does RLC matter, and does consistency?	Truncation; free-form summary; RLC at test time only; RLC at training and test time
A3	How strict should acceptance be?	$\tau \in \{0.7, 0.9, 1.0\}$
A4	Does the headroom filter matter?	No filter; 5% margin only; 3σ only; both
A5	How much does the teacher matter?	Selected teacher; runner-up teacher
A6	How much of the gain is contamination?	With and without decontamination, evaluated on both
A7	Is transfer caused by iteration?	Token-matched single-attempt trajectories from the same environments

context the agent sent to the model is a prefix of the training window that covers that turn. This is the property that Section 4.5 relies on. The weighted-loss and chat-template code has not yet been exercised on GPUs.

8 Full-Scale Evaluation Protocol

Benchmarks and budgets. We use the six AREX-2 benchmarks and budgets so that results are directly comparable: MLE-bench Lite [2] (Any Medal; 12 h per task with skills in context), the Frontier-CS Agent Track [14] (188 tasks; 5 h per task), and BrowseComp [26], HLE [18], GAIA [16], and DeepSearchQA [9] (300 inner turns and 1,500 turns overall). Each configuration runs three times; we report the mean and a 95% bootstrap confidence interval. For reference, AREX-2 reports 81.8, 70.7, 84.0, 52.6, 92.2, and 93.8 on these benchmarks, in the order listed.

Systems. (i) *Base*: Qwen3.8-27B in our scaffold with RLC, untrained. (ii) *AREX-2*: the public weights, run both in our scaffold and in the AREX-2 harness, which separates the effect of the model from the effect of the scaffold. (iii) *Mk33*: the full method.

Process metrics. Final scores do not show which capability improved. Following Equation (1), we also report: *productive rounds* T^*, the number of rounds before the first run of five rounds without an improving submission; *mean gain* $\bar{r}$, the mean normalized gain over improving rounds; *best-so-far AUC*, the area under the normalized best-so-far curve over the time budget; *recovery rate*, the share of regressions followed by a new best within three rounds; *wasted-turn share*, the share of test-time turns flagged by *R*, *N*, or *P*; and the *re-try rate* of Section 9.2.

Ablations. Each ablation changes one component (Table 4). A7 is the central test of the AREX-2 transfer claim: if single-attempt trajectories matched on token count produce the same deep-research gains, those gains come from additional agentic training rather than from learning to iterate. Following AREX-2, we also plot best-so-far score against time on Frontier-CS (with feedback) and against turn budget on BrowseComp (without feedback), and add these curves for each A2 variant, since context management should matter most late in long runs.

Table 5: Bench environments (seed 0). Raw scores from the isolated scorer; the reference is run with three seeds. Normalized scores map the baseline to 0 and the reference mean to 1.

Environment	Metric	Baseline	Reference runs
Regression	RMSE ↓	4.970	2.616, 2.614, 2.642
Sentiment	accuracy ↑	0.511	0.653, 0.657, 0.659
Forecast	MASE ↓	4.239	1.041, 1.019, 0.984
Knapsack	optimality gap ↓	0.321	0.0052, 0.0063, 0.0048

9 Small-Scale Experiments

The protocol of Section 8 needs GPU clusters and multi-hour runs. This section tests the mechanisms at a scale that fits one free GPU (a Colab T4) or a laptop CPU. All experiments use the scaffold, scorer, metrics, and statistics of the full protocol unchanged; what changes is the size of the models, the tasks, and the context window. Each experiment is released as a notebook or script that saves results after every run and resumes after a disconnect.

9.1 Setting

Environments. Four environments are generated from a seed and use only the Python standard library (Table 5): nonlinear regression (Friedman #1, RMSE), sentiment classification of synthetic reviews with negation and 3% label noise (accuracy), forecasting of 30 daily series with trend, weekly seasonality, and level shifts (MASE), and 24 hidden 0/1 knapsack instances (mean optimality gap). The agent sees a task statement, a weak baseline solution, and the public split; the held-out split, scorer, and reference solutions stay in the isolated scorer directory. Baseline and reference scores are measured by running the real scorer, and all four environments pass the headroom filter of Section 3. Knapsack has a known optimum, so its normalized score is capped near 1.02; the other three leave room above the reference. The experiments below use regression and knapsack. Different seeds generate different data, which gives a clean train/test split for Section 9.5: training environments use seeds 1 to 3 and all evaluations use seed 0.

Reproducing the overflow. The default window of 128K tokens is replaced by 8,000, with compaction at 75% and the last two rounds kept verbatim. A run of 8 rounds and 40 tool calls on these tasks exceeds this window, so every memory method is exercised; the long runs of Section 9.6 go to 40 rounds. Everything else in the scaffold is unchanged.

Statistics. Runs are paired by environment and repeat. For each pair of systems we report the mean paired difference with a seeded bootstrap 95% interval [7] and call a difference real only when the interval excludes zero.

9.2 Re-try rate

The process metrics of Section 8 measure progress. Context management exists to prevent a specific failure: forgetting what has already been tried. We measure it directly. Round $t \geq 2$ is a *re-try* if (a) the scored solution is identical, after whitespace normalization, to one submitted in an earlier round, or (b) the word set of its `hypothesis` has Jaccard similarity of at least 0.6 with the hypothesis of an earlier round that failed or did not improve on the best score. Hypotheses of fewer than three content words are not compared, and an idea that previously improved the score is never counted. The re-try rate is the share of rounds after the first

Table 6: Loss increase over the full cache (nats; lower is better), mean over 8 windows. Bold: lowest in the row. Intervals are in Figure 2.

		KV-cache eviction				Text level	
Kept	Model	H2O	Key-norm	Random	Streaming	Truncation	BM25
1,024	Qwen	**0.203**	0.304	0.259	0.284	0.288	0.288
1,024	Llama	**0.265**	0.376	0.456	0.470	0.291	0.327
2,048	Qwen	**0.120**	0.230	0.191	0.210	0.206	0.222
2,048	Llama	**0.156**	0.262	0.349	0.363	0.196	0.242
4,096	Qwen	**0.049**	0.104	0.079	0.141	0.133	0.125
4,096	Llama	**0.068**	0.103	0.144	0.262	0.131	0.122

Table 7: Paired comparisons at budgets 1K / 2K / 4K. "Better" or "worse" only when the bootstrap 95% interval of the paired difference excludes zero; otherwise "unclear".

Comparison	Qwen2.5-3B	Llama-3.2-3B
H2O vs random	unclear / better / better	better / better / better
H2O vs BM25 retrieval	better / better / better	unclear / better / better
H2O vs truncation	better / better / better	unclear / unclear / unclear
BM25 retrieval vs truncation	unclear / unclear / unclear	unclear / unclear / unclear
Key-norm vs random	worse / worse / worse	better / better / unclear
Streaming vs random	worse / unclear / unclear	unclear / unclear / worse

that are re-tries; lower is better. Rule (a) is exact; rule (b) misses re-tries that are reworded, so the rate is a lower bound.

9.3 Memory at equal cache size

Once a run outgrows its window, something must be forgotten. Forgetting can happen in the text, before the model reads it (truncation, the ledger, or retrieval that puts relevant old text back), or inside the model, by keeping the full history and compressing the key-value (KV) cache to a fixed budget. Before measuring task success, we compare the two on a metric that needs no agent.

Protocol. A window of 8,192 context tokens is followed by 1,024 continuation tokens, sampled from a corpus of about 95K tokens of LaTeX, Markdown, and Python. We measure the next-token loss on the continuation when only *budget* tokens of the context remain in the cache, relative to the full cache. KV methods read the whole context and then evict: random (a control, two seeds), streaming [27] (attention sinks and recent tokens), key-norm [6] (low-norm keys), and H2O [30] (accumulated attention). Text methods cut the context before the model reads it: truncation keeps the first 4 and the most recent tokens; retrieval keeps the first 4 and last 256 tokens plus the earlier chunks most relevant to them by BM25 [22], the same retriever as the agent's `retrieve` mode. Every method ends at budget plus continuation cache tokens, so memory is equal by construction. Models are Qwen2.5-3B-Instruct [20] and Llama-3.2-3B-Instruct [8] in float16, 8 windows each.

Findings. Tables 6 and 7 and Figure 2 support four statements. (1) *H2O is the most reliable method.* It has the lowest loss in every cell, beats random in 5 of 6 comparisons and BM25 retrieval in 5 of 6, and is never significantly worse than any method. Keeping half the context, it stays within 0.05 to 0.07 nats of the full cache. (2) *BM25 retrieval was not distinguishable from truncation* in any of the 6 comparisons. For

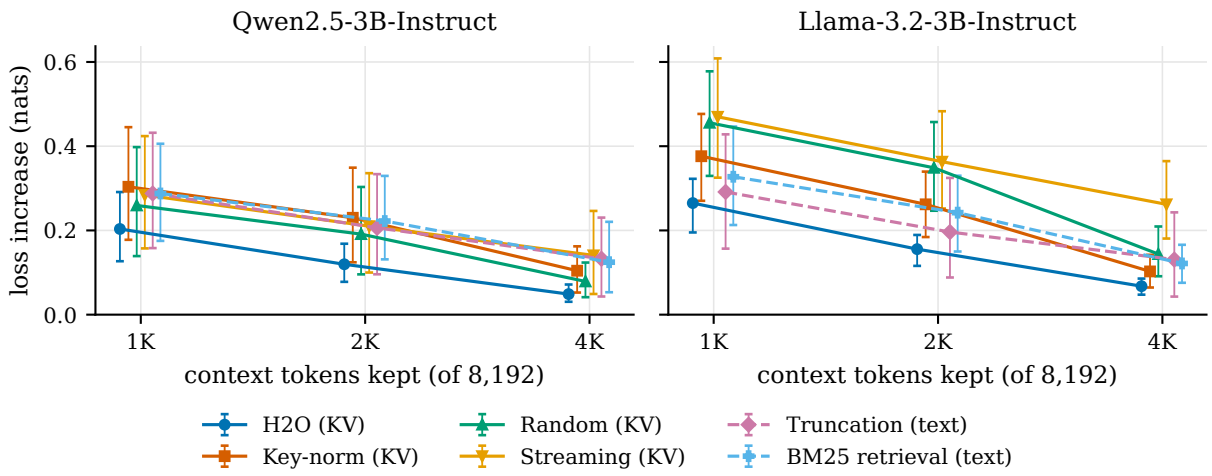


Figure 2: Loss increase over the full cache by memory budget, with bootstrap 95% intervals over 8 windows. Solid: KV-cache eviction. Dashed: text-level forgetting.

predicting the continuation of code and prose, the most recent text was as useful as keyword-matched older text. This is consistent with Corallo et al. [5], who find KV compression ahead of retrieval on broad tasks and retrieval still useful for narrow ones; an agent that needs one specific old fact, such as an error from round 2, is the narrow case, which Section 9.4 tests. (3) *Key-norm eviction depends on the model*: worse than random at every budget on Qwen2.5, better than random at 1K and 2K on Llama 3.2. A heuristic validated on one model family should be rechecked on another; why it fails on Qwen2.5 is not established here. (4) *Equal memory is not equal compute* (Figure 3). Text methods prefill only the kept tokens and took 1.6 to 1.8 s per measurement; KV methods read the whole context first (4.4 to 4.9 s; H2O 7.6 to 8.2 s, since it needs eager attention to accumulate scores). H2O's quality costs roughly 4.5 times the compute of retrieval at the same memory.

Kernels. The eviction step reads and gathers every layer's cache once per compression, so it is memory-bound. We implement it in Triton [25]. On a T4 (peak 320 GB/s) in float16, the key-norm kernel reaches 228 to 260 GB/s from 8K to 128K cached tokens, about 80% of peak and 5 to 7 times faster than computing norms in PyTorch, which first writes a float32 copy of the keys; the fused gather is 1.5 to 1.9 times faster than two `torch.gather` calls at 8K tokens and above. At 2K tokens a fixed launch cost of about 0.12 ms dominates and PyTorch is faster.

Limits. Two 3B models, one corpus, 8 windows, and two seeds for the random control; intervals are wide, and several "unclear" cells could resolve with more windows. Next-token loss is a proxy; Section 9.4 measures task success.

9.4 Task success under each memory method

This experiment asks whether the differences of Section 9.3 change what an agent solves. The agent runs the regression and knapsack tasks on one T4 (Kaggle) with five memory methods at an equal budget of 8,000 tokens: `truncate`, `rlc`, and `retrieve` (BM25 over dropped turns) at the text level; `kv:h2o`, which keeps the full text history and evicts the KV cache to 8,000 tokens; and `kv:full`, which forgets nothing up to the model's 32K limit and serves as an upper bound with four times the memory. Each run has up to 8 rounds, 40 tool calls, and 25 minutes, and each configuration is repeated three times (30 runs per grid). We

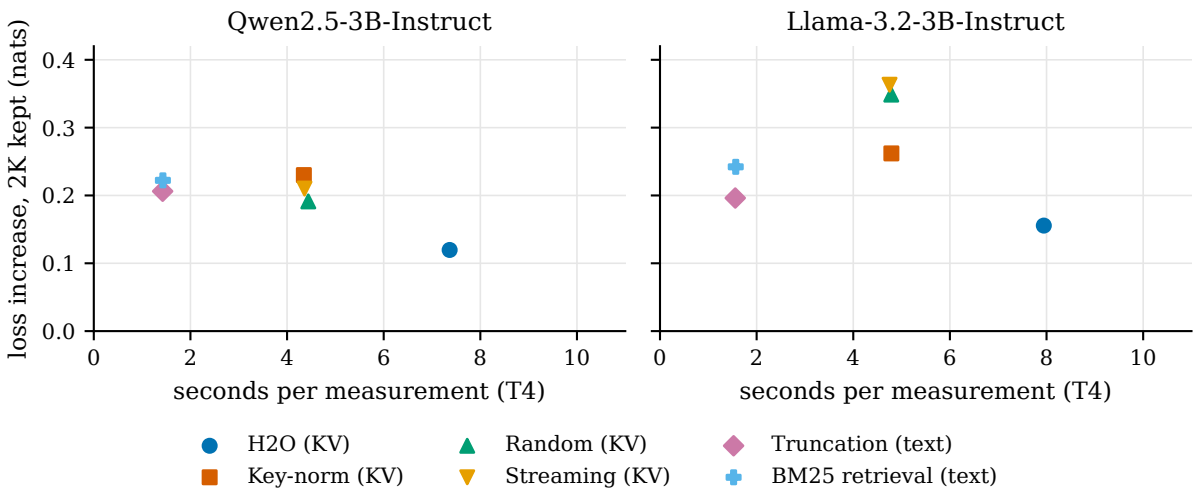


Figure 3: Quality against compute at equal memory (2,048 context tokens kept). Text methods read only the kept tokens; KV methods read all 8,192 first.

Table 8: Task success by memory method, 8 rounds per run. Best: mean best normalized score (baseline 0, reference 1; failed runs count as 0). 3B: two grids pooled, 12 runs per method. 7B: one grid, 6 runs per method, shown per task. All H2O runs with the 7B model crashed (see text).

	Qwen2.5-3B (fp16)		Qwen2.5-7B (4-bit)		
Method	Best	Re-try ↓	Regression	Knapsack	Re-try ↓
`truncate`	0.168	0.79	0.336	0.995	0.72
`rlc`	0.247	0.73	0.336	0.995	0.66
`retrieve`	0.207	0.74	0.454	0.995	0.78
`kv:h2o`	0.208	0.67	crashed (fp16 overflow)		
`kv:full` (4× memory)	0.214	0.74	0.000	0.995	0.72

ran the grid twice with Qwen2.5-3B-Instruct in float16 (the second time after scaffold fixes) and once with Qwen2.5-7B-Instruct quantized to 4-bit NF4. Comparisons are paired by task and repeat.

Findings. The grids do not separate the memory methods (Table 8). With the 3B model, `rlc` and `truncate` gave the same score in 10 of 12 pairs (mean difference $+0.08$, 95% interval $[0, 0.20]$); the two pairs that differ are the same regression environment and repeat in the two grids, so they are close to a single observation. The 3B model never improved on knapsack. With the 7B model, `rlc` and `truncate` gave the same score in all 6 pairs; every method reached 0.995 on knapsack, a ceiling, and the regression scores come from one repeat per method. Retrieval against truncation ($+0.06$, $[-0.50, 0.68]$) and the full cache against truncation are also indistinguishable.

The reason is that most runs never reach the regime the methods differ in. All modes send the same context until the first overflow. In 8 rounds the context overflowed late or not at all: with the 7B model, half of the `truncate` and `rlc` runs never compacted, and in the 3B grids H2O evicted any cache in only 7 of 12 runs. Once a run did overflow, few rounds remained for memory to matter. Task success at this length therefore measures the model, not its memory; the long runs below are the test of memory.

Table 9: Small-scale training configuration, shared by the three students (`configs/mk33/colab/`).

Item	Value
Student	Qwen2.5-1.5B-Instruct, float16, SDPA attention
Adapter	LoRA on all linear layers, rank 16, $\alpha = 32$, dropout 0.05; adapter weights in float32
Optimizer	AdamW, $\beta = (0.9, 0.95)$, no weight decay
Learning rate	Peak 2×10^{-4}, 5% warmup, cosine decay to 2×10^{-5}
Epochs; batch	3; 8 windows (micro-batch 1)
Sequence length	up to 10,240 tokens; gradient checkpointing
Hardware	one NVIDIA T4 (16 GB)

H2O crash. All six H2O runs with the 7B model failed at the first long prompt with non-finite sampling probabilities. H2O needs the attention weights, so it runs eager attention, which in float16 overflowed: query-key products above 65,504 become infinite and the softmax returns NaN. The 3B model stayed in range. The implementation now computes eager attention in float32 and casts the result back (a test reproduces the overflow by scaling the query and key projections), and the 7B grid will be rerun with the fix.

9.5 Loss masking: PALM against AREX-2's masking

PALM's central claim is that weighting turns by their contribution to progress trains a better long-horizon agent than masking only redundant, null, and polling turns, as AREX-2 describes. This experiment is ablation A1 at small scale, with AREX-2's rule as the main baseline.

Data. A teacher runs the regression and knapsack tasks with data seeds 1 to 3 (six training environments), two runs each, with RLC at an 8,000-token window, 8 rounds, and 40 tool calls. The teacher is an open-weight model run locally or served through an API; closed models are excluded because their terms forbid training on their outputs. A 3B teacher is unlikely to be enough: in Section 9.4 the 3B agent improved on only one of six task-repeats. Runs are accepted by the rule of Section 6.1 with $\tau = 0.5$ instead of 0.9, since a small teacher rarely reaches the reference; at most two runs are kept per environment. The audit is applied unchanged.

Three students. The accepted runs are turned into training windows three times, differing only in the turn weights: *PALM* (Equation (4), $\lambda = 0.5$), *AREX-2 masking* (weight 0 for *R*, *N*, *P* turns and 1 for every other turn, that is, PALM's zero-weight rules without progress-aware credit), and *uniform* (weight 1 on every assistant turn). Each set trains a LoRA adapter on Qwen2.5-1.5B-Instruct with identical settings (Table 9). The windows are built with the same 8,000-token RLC configuration and the same six tool schemas the agent uses at test time, so the training contexts are those the student will see.

Evaluation. The base model and the three students run the held-out tasks (seed 0) with RLC at the same budget as the teacher, three repeats each. The comparison that tests PALM against AREX-2 is the paired difference between the PALM and AREX-2 students; uniform against base shows what training on teacher runs does at all.

Status. The data pipeline, the three weightings, LoRA training, and evaluation are implemented and covered by tests, and the notebook is released. The experiment itself has not been run with a teacher strong enough to produce accepted runs, so no comparison of PALM with AREX-2's masking is reported here.

Table 10: Long runs: regression, 7B model on a laptop CPU, 5-hour and 40-round budget per run. Last gain: round of the last improvement. Compacted: round of the first compaction. s/call: median seconds per model call. Reused: mean share of each prompt that repeats the start of the previous prompt, from a replay of the contexts. Runs 1 and 4 are confounded (see text).

Run	Mode	Best	Rounds	Last gain	Compacted	s/call	Reused	Re-try ↓
1	`truncate`	0.000	21	–	8	585	39%	0.95
	`rlc`	0.000	40	–	10	61	87%	0.97
2	`truncate`	1.363	23	3	10	266	52%	0.91
	`rlc`	1.287	10	3	10	112	72%	0.78
3	`truncate`	0.000	24	–	13	283	51%	1.00
	`rlc`	1.558	40	27	10	93	85%	0.64
4	`truncate`	0.000	30	–	11	670	39%	0.97
	`rlc`	1.532	38	17	5	145	82%	0.84
5	`truncate`	0.000	18	–	7	599	42%	0.94
	`rlc`	1.558	40	11	12	182	72%	0.90

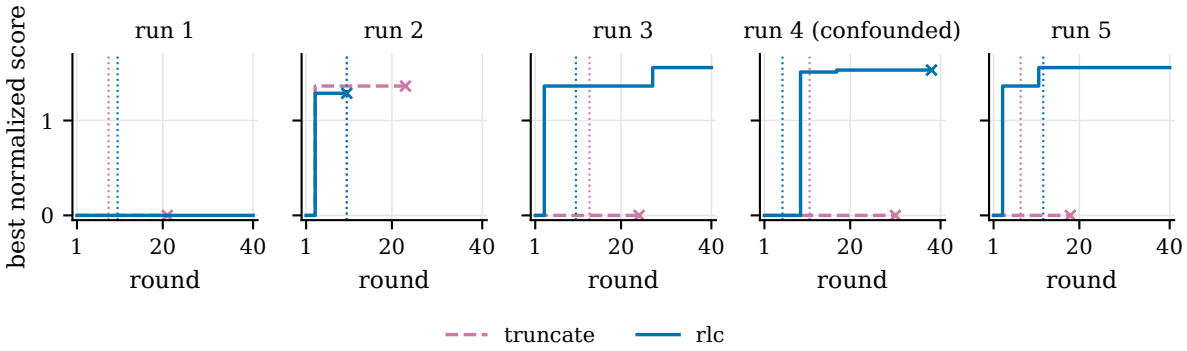


Figure 4: Best-so-far normalized score by round in the five long runs. Dotted lines: first compaction of each mode. A cross marks a run that ran out of time before 40 rounds.

9.6 Long runs

The claim behind RLC is about late rounds: an agent should keep improving after its early history no longer fits. We ran the regression task, whose score has no ceiling, for up to 40 rounds, 200 tool calls, and 5 hours per run, with `truncate` and with `rlc`, on a laptop CPU (Intel, 16 GB, no GPU; Qwen2.5-7B-Instruct, 4-bit, through Ollama with a 10,240-token context and an 8,000-token agent window). We made five such pairs. Between pairs we fixed problems that the runs exposed, so the pairs are not repeats of one configuration; within a pair both modes used the same code. Two pairs have confounds: in run 1 scikit-learn was not installed, so every solution that imported it failed, and in run 4 the `rlc` agent installed a gradient-boosting package halfway through, after which package installs were blocked. Table 10 and Figure 4 give the results; `scripts/mk33/long_run_stats.py` recomputes every number from the saved runs.

Speed. `rlc` needed less time per model call in all five pairs, by a factor of 2.4 to 9.6 (median 3.3). As a result it finished all 40 rounds in three pairs, while `truncate` never got past round 30 in 5 hours. The difference is not prompt length: replaying each run's contexts with the agent's own context builder, the mean prompt was 0.93 to 1.18 times as long under `truncate` as under `rlc`. It is consistent with prefix reuse.

Once the window is full, `truncate` drops the oldest turns at nearly every call (17 to 25 times per run, against 9 to 19 compactions for `rlc`, which also ran more rounds), and everything after the dropped turns shifts, so a server that caches the previous prompt, as Ollama does, can reuse only the part before the first change. `rlc` only appends between compactions, so on average 72 to 87% of each prompt was identical to the start of the previous one, against 39 to 52% for `truncate`. On a CPU, where reading the prompt dominates the cost of a call, this is the difference between minutes and seconds. Two caveats apply. We did not measure the server's cache hits directly, and output length was not controlled, so part of the gap may come from longer responses under `truncate`. The frozen prefix was introduced to keep training windows exact (Section 4.5); the speed is a side effect, but on local hardware it decides how many rounds a run gets.

Quality. Whether `rlc` leads to better solutions is not established. The three pairs without a confound split: in run 2 both modes made their only gain in round 3 and `truncate` ended slightly higher; in runs 3 and 5 `rlc` reached 1.558 and `truncate` stayed at the baseline. The two `rlc` wins have different causes. In run 5 the gain came in round 11, before the first compaction, while `truncate` spent 16 rounds on a syntax error (Section 9.7); that is a difference in the model's luck and speed, not memory. Run 3 is the one pair that matches the RLC hypothesis: `rlc` found a further gain in round 27, 17 rounds after its first compaction, with its lowest re-try rate (0.64), while `truncate` never edited the solution at all. One pair is not evidence. A test of quality needs repeated pairs on a fixed configuration, ideally with a server-independent budget in rounds rather than hours so that speed does not decide the outcome.

9.7 How small agents fail in long runs

The long runs and grids record every failed round with its error. Most failures had nothing to do with memory, and several were fixed in the scaffold between runs. We list them because they decide what a small-scale study can measure, and because each fix is now part of the released code.

- **Tool calls written as text.** After a compaction, the 7B model sometimes wrote its tool call as a JSON block in the message instead of a structured call. In run 2, 35 of the `rlc` agent's turns were lost this way, which is why it played only 10 rounds. The scaffold now recovers such calls from the text (16 recovered in run 3).
- **Escaped newlines in code.** The model wrote `\n` inside code it passed to `edit_file`, producing a file that does not parse. In run 5 every one of the `truncate` agent's 16 failed rounds had this error.
- **Repeating the same error.** In run 4 the `truncate` agent submitted code with the same undefined name 28 times in a row, although the error was in its context each time. A recent context is not enough when the model does not act on it.
- **Describing instead of editing.** In run 3 the `truncate` agent submitted 24 times without ever changing the solution, each time with a hypothesis describing a change it had not made (re-try rate 1.00). The scaffold now tells the agent when a submission is unchanged.
- **Wrong library interfaces.** In run 5 the `rlc` agent passed a parameter that the installed version of scikit-learn does not accept in 29 rounds.
- **Environment changes.** Missing packages failed all of run 1; in run 4 the agent installed a package mid-run and then tried others that were not available. Installs are now blocked and the package set is recorded with each run.

Re-try rates were high in every run (0.64 to 1.00): small agents spend most rounds repeating a known failure, whatever the memory method. This is the regime in which a training method such as PALM, which

removes repeated and null turns from the loss, should matter most, and it is a reason to expect that better memory alone will not make a small model improve over long runs.

9.8 Pilot run

Before the experiments above, we ran one hosted model through a free API tier on the knapsack task with 25 tool calls and 8 rounds per run, which allowed one pair of runs. The truncation run reached the optimum (normalized 1.017) in one round; the RLC run wrote and tested an exact dynamic-programming solver but used its 25 tool calls before submitting and scored 0. Until the first compaction both modes send nearly the same context, so this pair carries no evidence about RLC. It shows that the pipeline works end to end with a real model and that 25 tool calls is too tight a budget; the experiments above use 40. Earlier attempts with small local models did not finish: a reasoning model exceeded the per-call timeout, and a 3B model described edits instead of calling tools. Tool-calling reliability of small models is therefore a risk for the experiments above. The notebooks check that the model's tool calls parse before starting, and runs in which the model stops calling tools are reported rather than dropped.

10 Related Work

Iterative refinement. Self-Refine [13] and Reflexion [23] showed that language models can improve outputs using their own feedback or verbal reflection, over a few rounds and mainly at inference time. AREX-2 and this work train for refinement over hundreds of rounds.

Machine-learning engineering and coding agents. MLE-bench [2] measures agents on Kaggle-style tasks. AIDE [11] frames machine-learning engineering as tree search over code. SWE-agent [28] shows that the design of the agent-computer interface strongly affects performance, which motivates fixing the tool set in Section 4.1.

Agent fine-tuning. AgentTuning [29] and FireAct [4] fine-tune language models on agent trajectories. PALM additionally weights assistant turns by whether they contributed to progress, using rules computed from the trajectory log.

Context management. MemGPT [17] pages information between a bounded context and external memory. RLC is narrower: its memory has a fixed schema tied to submission rounds, the prefix is frozen between compactions, and the model is trained on exactly the compacted views it sees at test time.

KV-cache compression and retrieval. StreamingLLM [27] keeps attention sinks and recent tokens, H2O [30] keeps tokens with high accumulated attention, and Devoto et al. [6] keep low-norm keys. Chen et al. [3] study where KV compression fails. Corallo et al. [5] compare retrieval with task-aware KV compression on long-context question answering and find compression stronger on broad questions and retrieval still effective on narrow ones. MemDecay [15] evicts an agent's KV cache by the role of each region, such as the system prompt, plans, and tool outputs, and compares against recency- and attention-based retention, but not against text-level context management. The study in Section 9.3 re-implements published eviction policies; its additions are text-level and KV-level forgetting under strictly equal cache size with a random-eviction control and paired intervals, retrieval memory inside a self-improving agent whose training windows reproduce the retrieved context, and the task-level comparison of Section 9.4.

Reward hacking. Optimizing a measurable proxy can diverge from the intended objective [24]. Scorer isolation, canary tests, and the audit in Section 3.4 address this in verifiable environments.

11 Limitations

- **Small scale.** The experiments use 3B and 7B models, two synthetic task types, an 8K window, and runs of tens of rounds. They test whether the mechanisms work when the overflow problem is reproduced at small scale, not whether they reach AREX-2's scores or behave the same at 27B parameters, 128K tokens, and 12-hour runs. The full-scale protocol (Section 8) has not been run.
- **Small samples.** Three repeats per configuration and eight windows in the memory study give wide intervals; differences reported as "unclear" may be real effects too small to detect. The long runs are five pairs on one task, with fixes between pairs and two confounded pairs; they support the speed result, which held in every pair, but not a claim about solution quality.
- **Speed mechanism.** The prefix-reuse explanation of the long-run speedup rests on a replay of the contexts, not on the server's cache statistics, and response length was not controlled. The size of the effect depends on the server and hardware; on a GPU, where prompt processing is cheaper, it should be smaller.
- **Proxy tasks.** The bench tasks are small and synthetic. They have isolated scorers and real headroom, but they are not MLE-bench or Frontier-CS tasks, and results may not transfer.
- **PALM not yet tested.** The comparison of PALM with AREX-2's masking (Section 9.5) has not been run, so this report makes no claim that PALM trains a better agent. Small teachers produce few acceptable trajectories, and the acceptance threshold must be lowered for them.
- **Heuristic thresholds.** The Jaccard cutoffs, the windows W_R and M, and the acceptance thresholds are reasoned defaults rather than tuned values. The re-try rate's idea match misses reworded repeats, so it is a lower bound.
- **Incomplete provenance.** S^+ credits surviving edits and evidence one hop away. It misses insights that shape later decisions without shared tokens, such as learning that an approach is a dead end; such turns receive weight λ.
- **Incomplete implementation.** The free-form summary baseline of A2, the repository crawler, canary-test generation, and the LLM-judge prompt for A1 are not yet implemented, and the human-label check of the PALM rules (Section 5) has not been run.

12 Conclusion

AREX-2 shows that training on long improvement trajectories is a promising route to agents that improve with more time. Building on that result requires a recipe others can run. Mk33 provides one: explicit environment filters and thresholds, a context manager that keeps failures visible for runs of any length and yields exact training windows, deterministic progress-aware loss masking, and a complete training configuration, each paired with an ablation and implemented in tested, open code. Because the full recipe is expensive, every experiment also runs at a scale that fits one free GPU. At that scale, attention-based KV eviction preserved next-token quality best at equal memory, and keyword retrieval did not beat truncation. In 40-round runs on a laptop, RLC's frozen prefix made each model call 2.4 to 9.6 times faster than truncation by letting the server reuse its cache, so RLC runs got far more rounds in the same time. Short runs did not separate memory methods, and whether RLC improves solutions, and whether PALM trains better agents than AREX-2's

masking, remain open; both need repeated long runs and a stronger teacher than free hardware easily provides. The failures we observed, from tool calls written as text to the same error repeated dozens of times, suggest that for small agents the main obstacle to long runs is acting on the history, not keeping it.

Code Availability

The reference implementation, configuration files, tests, and the notebooks and scripts for every experiment in Section 9 are available at `https://github.com/dulguuntsatsralt-cell/Mk33`. The raw results are in `docs/mk33/results/`: the memory study, the task-success grids (`task-success/`), and the long runs with their full trajectories (`long-runs/`). `scripts/mk33/long_run_stats.py` recomputes the long-run numbers and `scripts/mk33/paper_figures.py` regenerates the figures.

Acknowledgments

Mk33 builds directly on the AREX-2 method of Qian et al. [19]. We thank the AREX-2 authors for describing their recipe in detail and for releasing their evaluation suite, which made this reference implementation possible. They were not involved in this work, and any errors in interpreting or extending their method are the author's own. The author used an AI assistant to help draft text and code; the author reviewed the content and is responsible for it.

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

[2] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.

[3] Alex Chen, Renato Geh, Aditya Grover, Guy Van den Broeck, and Daniel Mingyi Israel. The pitfalls of KV cache compression. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026.

[4] Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. FireAct: Toward language agent fine-tuning. *arXiv preprint arXiv:2310.05915*, 2023.

[5] Giulio Corallo, Orion Weller, Fabio Petroni, and Paolo Papotti. Beyond RAG: Task-aware KV cache compression for comprehensive knowledge reasoning. *arXiv preprint arXiv:2503.04973*, 2025.

[6] Alessio Devoto, Yu Zhao, Simone Scardapane, and Pasquale Minervini. A simple and effective L_2 norm-based strategy for KV cache compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

[7] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[8] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[9] Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. DeepSearchQA: Bridging the comprehensiveness gap for deep research agents. *arXiv preprint arXiv:2601.20975*, 2026.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[11] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.

[12] Shuqi Lu, Chaofan Li, Kun Luo, Zhang Zhang, Hui Wang, Hongwang Xiao, Lei Xiong, Jiahao Wang, Sen Wang, Xiyan Jiang, et al. AREX: Towards a recursively self-improving agent for deep research. *arXiv preprint arXiv:2607.21461*, 2026.

[13] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

[14] Qiuyang Mang, Wenhao Chai, Zhifei Li, Huanzhi Mao, Shang Zhou, Alexander Du, Hanchen Li, Shu Liu, Edwin Chen, Yichuan Wang, et al. FrontierCS: Evolving challenges for evolving intelligence. *arXiv preprint arXiv:2512.15699*, 2025.

[15] Venkatesha Matam and Keon Kim. MemDecay: Region-aware KV cache eviction for efficient LLM agent inference. *arXiv preprint arXiv:2607.10582*, 2026.

[16] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.

[17] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[18] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity's last exam. *arXiv preprint arXiv:2501.14249*, 2025.

[19] Hongjin Qian, Chaofan Li, Kun Luo, Wenqing Wei, Jianlyu Chen, Shuqi Lu, Yuyang Hu, Hongwang Xiao, Hui Wang, Chaozhuo Li, Qiwei Ye, Zhicheng Dou, Defu Lian, and Zheng Liu. AREX-2: Advancing self-improving agents through long-horizon reflective tasks. *arXiv preprint arXiv:2609.38288*, 2026.

[20] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[21] Qwen Team. Qwen3.8. https://github.com/QwenLM/Qwen3.8, 2026.

[22] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[23] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[24] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022.

[25] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.

[26] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

[27] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

[29] Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling generalized agent abilities for LLMs. *arXiv preprint arXiv:2310.12823*, 2023.

[30] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.

A Default Configuration

The default configuration of the reference implementation (`configs/mk33/default.yaml`), abridged to the keys discussed in this report:

```
filter:    {sigma_k: 3.0, rel_margin: 0.05, min_reference_runs: 3,
            judge_max_baseline: 0.6, judge_min_reference: 0.95}
decontam:  {enabled: true, n: 13, threshold: 0.5}
audit:     {literal_min_len: 32, literal_short_len: 8,
            literal_short_count: 3, canary_gap: 0.2}
tools:     {bash_timeout_s: 600, max_output_tokens: 8000, max_wait_s: 1800}
rlc:       {mode: rlc, window: 128000, theta: 0.75, keep_rounds: 2,
            ledger_cap: 16000}
budget:    {ml: {max_tool_calls: 400, max_wall_s: 43200},
            judge: {max_tool_calls: 400, max_wall_s: 18000}}
selection: {tau: {ml: 0.9, judge: 0.95}, min_rounds: 3,
            require_iteration: true, max_per_env: 2}
palm:      {mode: palm, lam: 0.5, window: 20, jaccard: 0.9, m_rounds: 3,
            evidence_window: 50, ngram: 8}
sft:       {model: Qwen/Qwen3.8-27B, max_len: 131072, learning_rate: 1.0e-5,
            min_lr_ratio: 0.1, warmup_ratio: 0.03, weight_decay: 0.1,
            adam_beta1: 0.9, adam_beta2: 0.95, epochs: 2, global_batch: 64,
            max_grad_norm: 1.0, bf16: true}
```

B Data Formats

- **Environment specification**: task statement, workspace directory, scorer command (with `{solution}` and `{scorer_dir}` placeholders), script hash, metric direction, baseline score, reference scores, and dataset URLs, hashes, and problem identifiers for decontamination. The scorer prints `{"score": <float>}` as its last line of output.
- **Trajectory**: one JSON object per line. Each turn records reasoning, one tool call, the observation, the workspace hash, the diff for edits, and, for submissions, the score, normalized score, hypothesis, lesson, error signature, and a snapshot of the solution's text files.
- **Training window**: chat messages with tool calls, and one weight per message: the PALM weight for assistant targets and null for context. Deep-research data is converted to the same format with weight 1 on assistant messages.